

REST API SPECIFICATION • v1.0

AsanShipping Developer API Documentation

Enterprise Integration Manual for Custom Storefronts, Mobile Apps, Inventory Ledgers, Order Ingestion, Courier Milestone Tracking & Webhook Subscriptions.

ENVIRONMENT BASE URLS

<https://merchant-api.asanshipping.com/api/v1>

PROTOCOL & AUTH

HTTPS / Bearer API Key

DOCUMENTATION DATE

September 2026 (v1.0)

1. System Overview & Architecture

The AsanShipping Developer API enables merchants and technology partners to programmatically access catalog, inventory ledger, canonical order placement, courier tracking, and real-time webhook subscriptions.

Enterprise Multi-Tenancy & Isolation: Every API key is strictly scoped to a single merchant organization. Cross-tenant access is prohibited at the database engine level.

Base URLs

Environment	Base URL	Purpose
Live Production	<code>https://merchant-api.asanshipping.com/api/v1</code>	Production store integrations & live order processing
Local / Sandbox	<code>http://localhost:5000/api/v1</code>	Local staging, CI/CD pipelines & test suites

Standard Request & Response Headers

Header Name	Type	Description
<code>Authorization</code>	string	Bearer authentication token: <code>Bearer as_live_...</code>
<code>X-API-Key</code>	string	Alternative header to pass secret or storefront API keys
<code>Idempotency-Key</code>	string (UUIDv4)	Guarantees atomic, non-duplicate execution on mutating POST/PATCH endpoints
<code>X-Request-Id</code>	string	Unique telemetry identifier returned in every response header for tracing
<code>X-Cache-Lookup</code>	string	Set to <code>IDEMPOTENT_HIT</code> when returning a cached 24h replay response

Authentication & Permission Scopes

API keys are created in the Merchant Portal under **Settings → Developer & API**.

Scope Key	Description
<code>products:read</code>	View catalog items, prices, variants, and image URLs.
<code>products:write</code>	Create/edit products, manage SKUs, and upload WebP images to Backblaze B2.
<code>inventory:read</code>	Query real-time stock balances across warehouses.
<code>inventory:write</code>	Submit authoritative adjustments (restocks/damages) and transfers.
<code>orders:read</code>	Query canonical orders, customer contacts, and consignments.
<code>orders:write</code>	Ingest orders with automated phone normalization and destination checks.
<code>orders:cancel</code>	Cancel unbooked orders with automatic inventory restocking.
<code>tracking:read</code>	Public and authenticated parcel milestone tracking timeline.
<code>webhooks:manage</code>	Register, inspect, and test webhook subscriptions.

2. Reliability, Idempotency & Errors

Idempotency Protocol (24–Hour Window)

To eliminate duplicate charges and repeated order creation during network dropouts, AsanShipping implements an enterprise Idempotency Protocol on mutating endpoints (`POST /orders` , `POST /inventory/adjustments` , `POST /products`).

How to use: Send a unique UUIDv4 in the `Idempotency-Key` header. If a network timeout occurs and your client resends the identical request within 24 hours, the server returns the cached response with the `X-Cache-Lookup: IDEMPOTENT_HIT` header without executing duplicate side-effects.

Rate Limits

Environment Tier	Rate Limit	Burst Allowance	429 Penalty Window
Live Production (<code>as_live_...</code>)	120 requests / min	30 requests	Check <code>Retry-After</code> header
Sandbox Testing (<code>as_test_...</code>)	60 requests / min	15 requests	Check <code>Retry-After</code> header

Unified Error Protocol

All client and server errors adhere to the standard structured error schema:

```
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "Requested quantity (5) exceeds available stock (2) for SKU 'TS-BLK-M'",
    "details": { "sku": "TS-BLK-M", "requested": 5, "available": 2 },
    "timestamp": "2026-09-26T14:00:00.000Z"
  },
  "meta": {
    "requestId": "req_01J8EF89X...",
    "documentation": "https://asanshipping.com/developers#rate-limits"
  }
}
```

Error Code Directory

Status	Error Code	Meaning & Resolution
401	<code>UNAUTHORIZED</code>	Missing or invalid API key in Authorization header.
403	<code>FORBIDDEN_INSUFFICIENT_SCOPE</code>	API key lacks the required permission scope.
400	<code>INVALID_IDEMPOTENCY_KEY</code>	Malformed or empty Idempotency-Key header on mutating route.
409	<code>IDEMPOTENCY_PAYLOAD_MISMATCH</code>	Reused Idempotency-Key with different request payload.
429	<code>RATE_LIMIT_EXCEEDED</code>	Exceeded allowed requests per minute.
422	<code>INSUFFICIENT_STOCK</code>	Requested quantity exceeds available ledger balance.
400	<code>ORDER_ALREADY_DISPATCHED</code>	Cannot cancel or modify an order already booked with courier.

3. Store Profile & Catalog Products

GET /store

Scope: Valid API Key

Retrieve merchant profile, operational currency, timezone, and warehouse fulfillments.

```
// Response 200 OK
{
  "success": true,
  "data": {
    "id": "66ce3698ff8e3e449a622b10",
    "name": "Luxe Couture Official",
    "currency": "PKR",
    "timezone": "Asia/Karachi",
    "warehouses": [
      { "id": "wh-khi-01", "name": "Karachi Central Hub", "city": "Karachi", "isDefault": true }
    ],
    "supportedPaymentMethods": ["COD", "Prepaid"],
    "supportedCouriers": ["TCS", "Trax", "PostEx", "Leopards", "M&P"]
  }
}
```

GET /products

Scope: products:read

Query paginated list of catalog products. Supports query parameters: `page`, `limit`, `search`, `sku`, `status`, `stockStatus`.

POST /products

Scope: products:write [Idempotent]

Create a catalog product with variant pricing, initial inventory allocation, and dimensional weight.

```
// Request Body JSON
{
  "sku": "HD-EMR-XL",
  "title": "Emerald Heavyweight Hoodie",
  "price": 4999,
  "compareAtPrice": 5999,
  "weightGrams": 650,
  "inventory": { "warehouseId": "wh-khi-01", "quantity": 40 }
}
```

POST /products/:id/images

Scope: products:write

Upload a product image (JPG/PNG/WebP, up to 10MB). Automatically compressed and stored on Backblaze B2.

4. Authoritative Inventory & Canonical Orders

GET /inventory

Scope: inventory:read

Retrieve authoritative on-hand, reserved, and available stock quantities per warehouse node.

POST /inventory/adjustments

Scope: inventory:write [Idempotent]

Submit atomic delta adjustments (+50 restock, -2 damage) with audit trail.

```
{
  "sku": "TS-BLK-M",
  "warehouseId": "wh-khi-01",
  "quantityDelta": 25,
  "reason": "RESTOCK",
  "referenceId": "P0-2026-992"
}
```

POST /orders

Scope: orders:write [Idempotent]

Submit a canonical customer order. Automatically validates destination city & subarea against AsanShipping directory, normalizes mobile numbers to E.164 (+92), reserves inventory, and logs an idempotent transaction.

```
// Request Body JSON
{
  "orderNumber": "WEB-2026-9041",
  "customer": { "name": "Bilal Ahmed", "phone": "03001234567", "email": "bilal@example.com" },
  "shippingAddress": { "address1": "House 42, Street 5, DHA Phase 6", "city": "Lahore", "subarea": "DHA Phase 6" },
  "items": [{ "sku": "TS-BLK-M", "quantity": 2, "unitPrice": 2499 }],
  "paymentMethod": "COD",
  "totalAmount": 5248
}
```

POST /orders/:id/cancel

Scope: orders:cancel

Cancel an unbooked order and automatically restock reserved items back to warehouse ledger.

5. Delivery Locations & Address Reader Tool

GET `/locations/cities`Scope: `locations:read` / `Storefront`

Access all available Pakistani delivery cities, provinces, logistics tiers, and sub-area coverage zones. Ideal for checkout city dropdowns and destination validation.

```
// Query Parameters: ?province=Punjab&search=Lahore&includeAreas=true
{
  "success": true,
  "data": [
    {
      "name": "Lahore",
      "province": "Punjab",
      "tier": "Tier 1",
      "areasCount": 42,
      "areas": ["Johar Town", "Gulberg", "DHA Phase 5", "Model Town", "Bahria Town"]
    },
    {
      "name": "Karachi",
      "province": "Sindh",
      "tier": "Tier 1",
      "areasCount": 56,
      "areas": ["Clifton", "DHA Phase 6", "Gulshan-e-Iqbal", "North Nazimabad"]
    }
  ]
}
```

POST `/tools/parse-address`Scope: `tools:execute` / `orders:write`

Intelligently parse unstructured customer text into normalized Pakistani phone numbers (+923...), standardized delivery cities, sub-areas, and clean street addresses with confidence scoring. Supports single strings and high-speed bulk arrays (up to 100 items per request).

```
// Request Body JSON (Single or Bulk)
{
  "addresses": [
    "Ali Khan, 0300-1234567, Flat 4B, Johar Town, Lhr",
    "Fatima Tariq, +92 321 9876543, House 12, Gulshan-e-Iqbal, Karachi"
  ]
}

// Response JSON
{
  "success": true,
  "data": {
    "results": [
      {
        "originalText": "Ali Khan, 0300-1234567, Flat 4B, Johar Town, Lhr",
        "detectedPhone": "03001234567",
        "phoneNormalized": "+923001234567",
        "detectedCity": "Lahore",
        "detectedProvince": "Punjab",
        "detectedSubArea": "Johar Town",
        "cleanedStreetAddress": "Ali Khan, Flat 4B",
        "confidence": { "city": 98, "subArea": 100, "phone": 100, "overall": 99 }
      }
    ],
    "totalProcessed": 1
  }
}
```

6. Public Tracking & Webhooks

GET /tracking/:trackingNumber

Scope: tracking:read / Public

Public lookup endpoint returning the complete chronological journey milestones and COD balance.

```
{
  "success": true,
  "data": {
    "trackingNumber": "ASN-PK-8921849",
    "courier": "Trax Logistics",
    "status": "OUT_FOR_DELIVERY",
    "originCity": "Karachi",
    "destinationCity": "Lahore",
    "codAmount": 5248,
    "milestones": [
      { "status": "BOOKED", "location": "Karachi Hub", "timestamp": "2026-09-25T14:20:00Z" },
      { "status": "IN_TRANSIT", "location": "Lahore Sort Facility", "timestamp": "2026-09-26T06:45:00Z" },
      { "status": "OUT_FOR_DELIVERY", "location": "DHA Lahore Branch", "timestamp": "2026-09-26T10:15:00Z" }
    ]
  }
}
```

Webhooks & HMAC-SHA256 Signature Verification

Every webhook payload sent to your server contains the header `X-Asan-Signature: sha256={timestamp}.{hmacHex}`.

Node.js Verification Implementation

```
import crypto from 'crypto';

export function verifyAsanWebhook(rawBodyBuffer, signatureHeader, webhookSecret) {
  const parts = signatureHeader.split('.');
  const timestamp = parts[0].replace('sha256=', '');
  const signature = parts[1];

  // Prevent replay attacks (reject payloads older than 5 minutes)
  if (parseInt(timestamp) < Math.floor(Date.now() / 1000) - 300) {
    throw new Error('Webhook timestamp expired');
  }

  const expected = crypto
    .createHmac('sha256', webhookSecret)
    .update(`${timestamp}.${rawBodyBuffer.toString('utf8')}`)
    .digest('hex');

  return crypto.timingSafeEqual(Buffer.from(signature, 'hex'), Buffer.from(expected, 'hex'));
}
```

Supported Webhook Topics

Topic	Description
order.created	Fired immediately after an order is accepted into the system.
order.updated	Fired when customer details, delivery address, or consignments are updated.
order.cancelled	Fired when an order is cancelled and items are returned to inventory.
tracking.milestone_updated	Fired when courier pushes a new tracking scan.
inventory.low_stock	Fired when stock drops below safety reorder threshold.